BÁO CÁO THỰC HÀNH

BÀI 04: HỒI QUY TUYẾN TÍNH & HỒI QUY LOGISTIC

(LINEAR REGRESSION & LOGISTIC REGRESSION)

Sinh viên: Huỳnh Triệu An | Mã sinh viên: 65130011

Môi trường triển khai: Python 3.10+ / Visual Studio Code | Dữ liệu: Housing.csv & diabetes.csv

I. TỔNG QUAN BÀI TOÁN & THUẬT TOÁN

1. Mục tiêu và tổ chức thực nghiệm

Thực hành gồm hai bài toán có giám sát: (1) hồi quy đa biến dự đoán biến liên tục price; (2) hồi quy Logistic dự đoán xác suất Outcome = 1. Mục tiêu là tổ chức quy trình huấn luyện có thể tái lập, tiền xử lý đúng thứ tự và phân tích nhiều chỉ số thay vì chỉ quan sát một điểm số.

- Dùng 80% dữ liệu để huấn luyện và 20% kiểm thử; random_state = 42; riêng Diabetes chia bảo toàn tỷ lệ nhãn bằng stratify=y.
- Chỉ học tham số tiền xử lý (median và mean/std) trên Train; dữ liệu Test chỉ được transform, tuyệt đối không fit.
- Bài toán nhà ở: MAE, RMSE, R^2, đồ thị phần dư và phân tích hệ số trọng số.
- Bài toán tiểu đường: Confusion Matrix, Precision, Recall, F1-Score, ROC-AUC và ngưỡng 0.3/0.4/0.5/0.6.

2. Nguyên lý toán học

Hồi quy tuyến tính đa biến: $\hat{y} = w_0 + \Sigma_j\, w_j x_j$; hàm mất mát MSE = $(1/n)\Sigma_i(y_i - \hat{y}_i)^2$. Bài toán bình phương tối thiểu có thể giải bằng công thức nghiệm chuẩn khi thích hợp hoặc bằng phương pháp đại số ổn định số (chẳng hạn SVD trong thư viện); Gradient Descent cập nhật $w \leftarrow w - \alpha\nabla J(w)$.

Hồi quy Logistic: $p(y=1|x) = \sigma(w^Tx + b)$, với $\sigma(z) = 1/(1+e^{(-z)})$. Hàm Log-loss trung bình: $-(1/n)\Sigma[y \log(p) + (1-y)\log(1-p)]$. Điều chuẩn L2 hạn chế trọng số quá lớn; ngưỡng cắt t chuyển xác suất thành nhãn: $\hat{y} = 1$ nếu $p \geq t$.

Với nhãn dương là "mắc bệnh": Precision = TP/(TP+FP), Recall = TP/(TP+FN), F1 = 2PR/(P+R). ROC-AUC đo khả năng xếp hạng giữa ca dương và âm qua mọi ngưỡng.

3. Công cụ và bộ dữ liệu

Dữ liệu	Quan sát	Cột	Biến mục tiêu	Tách Train / Test
Housing.csv	545	13	price	436 / 109
diabetes.csv	768	9	Outcome	614 / 154

Thư viện: numpy, pandas, scikit-learn, matplotlib, seaborn, Jupyter; môi trường Visual Studio Code với môi trường ảo ml_lab_env. Tập Housing có 13 cột tổng cộng (gồm price); sau mã hóa số đặc trưng đầu vào là 13. Tập Diabetes có 8 đặc trưng và 1 nhãn.

II. CHƯƠNG TRÌNH CÀI ĐẶT VÀ KẾT QUẢ THỰC NGHIỆM

1. Nhiệm vụ 1: Hồi quy tuyến tính đa biến – Housing

1.1. Tiền xử lý và mã nguồn chính

Các cột mainroad, guestroom, basement, hotwaterheating, airconditioning và prefarea được ánh xạ yes → 1, no → 0. Biến furnishingstatus được One-Hot Encoding (drop_first=True) để bỏ nhóm tham chiếu. Tập dữ liệu này không có ô trống theo isna().

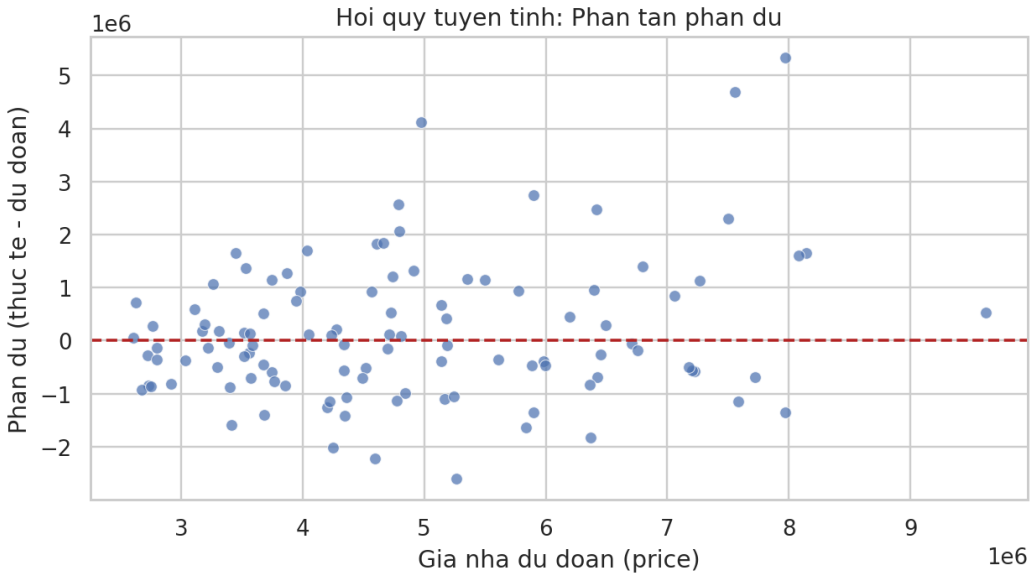
```
X_raw = df.drop(columns=['price']).copy()
for col in binary_cols:
    X_raw[col] = X_raw[col].map({'yes': 1, 'no': 0})
X_encoded = pd.get_dummies(X_raw, columns=['furnishingstatus'],
                           drop_first=True, dtype=int)
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
model = LinearRegression().fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
```

1.2. Chỉ số đánh giá

Chỉ tiêu	Giá trị thực nghiệm	Nhận xét
MAE	970,043.40	Sai lệch tuyệt đối trung bình
RMSE	1,324,506.96	Nhạy với sai số lớn
R^2 (Test)	0.6529	Giải thích khoảng 65,29% phương sai giá
R^2 (Train)	0.6859	Cao hơn R^2 Test khoảng 0,033

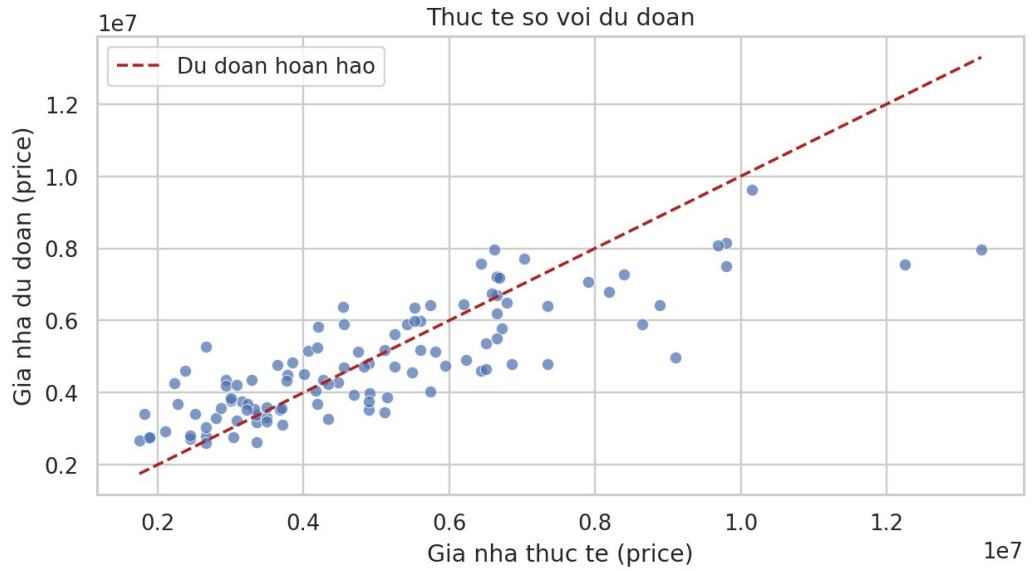
Biến price được đánh giá theo đơn vị số gốc trong CSV. Đề thực hành gọi đơn vị này là USD nhưng trong tệp dữ liệu không có cột metadata xác nhận đơn vị tiền tệ; không thực hiện quy đổi.

1.3. Biểu đồ kiểm tra chất lượng mô hình



Hình 1. Phần dư trên 109 mẫu kiểm thử của mô hình hồi quy tuyến tính.

Phần dư phân bố cả hai phía đường 0; vẫn có một số điểm lệch lớn (trên 4 triệu đơn vị price) tại vùng dự đoán giá cao. Độ phân tán có xu hướng lớn hơn khi giá dự đoán tăng, gợi ý khả năng phương sai không đồng nhất; từ một đồ thị chưa thể khẳng định chắc chắn vi phạm giả định.



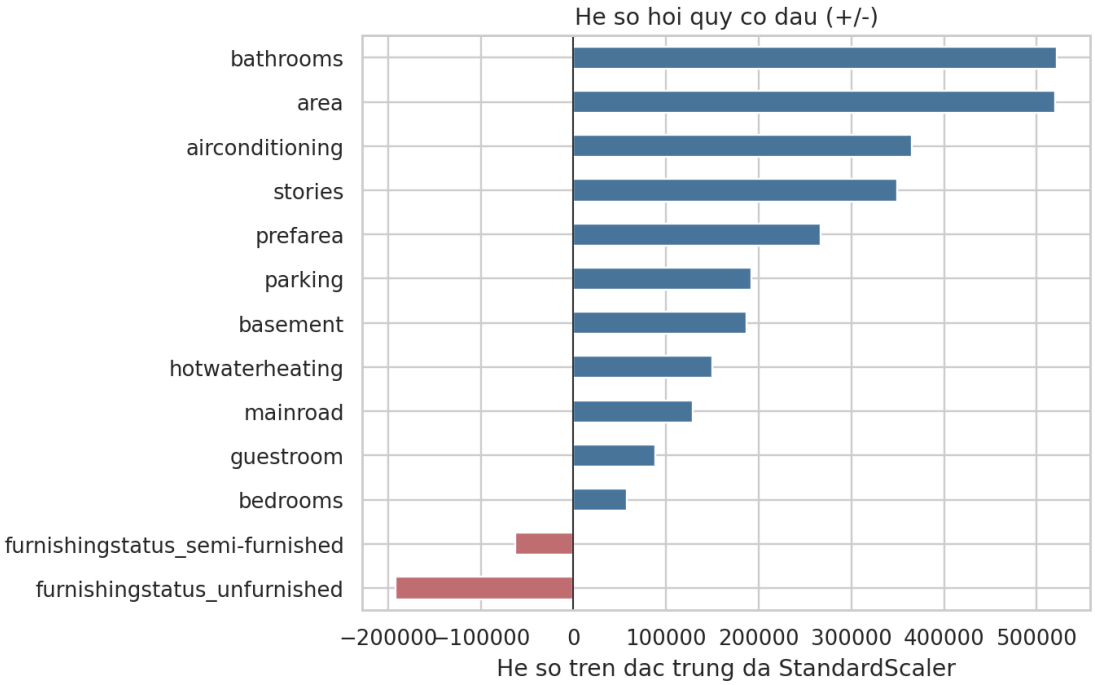
Hình 2. So sánh giá nhà thực tế và dự đoán; đường đứt thể hiện dự đoán lý tưởng.

Các điểm không hoàn toàn bám đường y = x: mô hình học được quan hệ tổng quát nhưng vẫn dự đoán sai đáng kể với một số căn nhà có giá cao, phù hợp với RMSE lớn hơn MAE.

1.4. Ý nghĩa hệ số đặc trưng

Đặc trưng	Hệ số sau chuẩn hóa	Diễn giải dấu
bathrooms	521,879	Dương
area	519,552	Dương
airconditioning	365,157	Dương

Đặc trưng	Hệ số sau chuẩn hóa	Diễn giải dấu
stories	349,251	Dương
furnishingstatus_unfurnished	-192,016	Âm so với nhóm furnished



Hình 3. So sánh hệ số hồi quy theo từng đặc trưng sau StandardScaler.

Các hệ số bathrooms và area có trị tuyệt đối lớn nhất (~522 nghìn và ~520 nghìn). Chúng phản ánh mức thay đổi dự báo khi đặc trưng đã chuẩn hóa tăng một đơn vị, giữ các biến khác cố định. Không coi hệ số là tác động nhân quả; biến nhị phân khi chuẩn hóa cần được diễn giải theo mức thay đổi một độ lệch chuẩn, không phải tự động từ 0 lên 1.

2. Nhiệm vụ 2: Hồi quy Logistic – Diabetes

2.1. Phát hiện dữ liệu thiếu ngụy trang dưới số 0

Không có giá trị NaN trực tiếp trong CSV, nhưng 0 là bất hợp lý đối với năm chỉ số sinh học dưới đây. Chỉ xét các mẫu Train khác 0 để tính median rồi dùng chính giá trị đó thay số 0 trên cả Train và Test. Không thay số 0 ở Pregnancies vì giá trị này có ý nghĩa hợp lệ.

Cột	Số giá trị 0 toàn bộ dữ liệu	Median tính trên Train
Glucose	5	117
BloodPressure	35	72
SkinThickness	227	29
Insulin	374	125
BMI	11	32.4

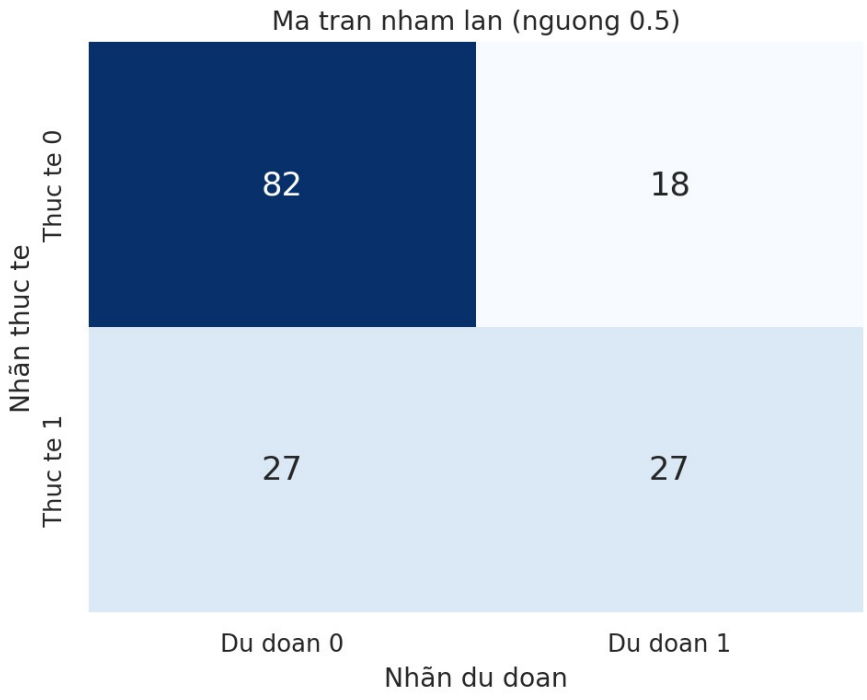
```
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42, stratify=y)
X_train, X_test = X_train.copy(), X_test.copy()
for col in zero_invalid_cols:
   median_train = X_train.loc[X_train[col] != 0, col].median()
   X_train[col] = X_train[col].replace(0, median_train)
   X_test[col] = X_test[col].replace(0, median_train)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
model = LogisticRegression(C=1.0, solver='lbfgs',
                           max_iter=1000, random_state=42)
model.fit(X_train_scaled, y_train)  # L2 la gia tri mac dinh
```

2.2. Kết quả phân loại tại ngưỡng t = 0,5

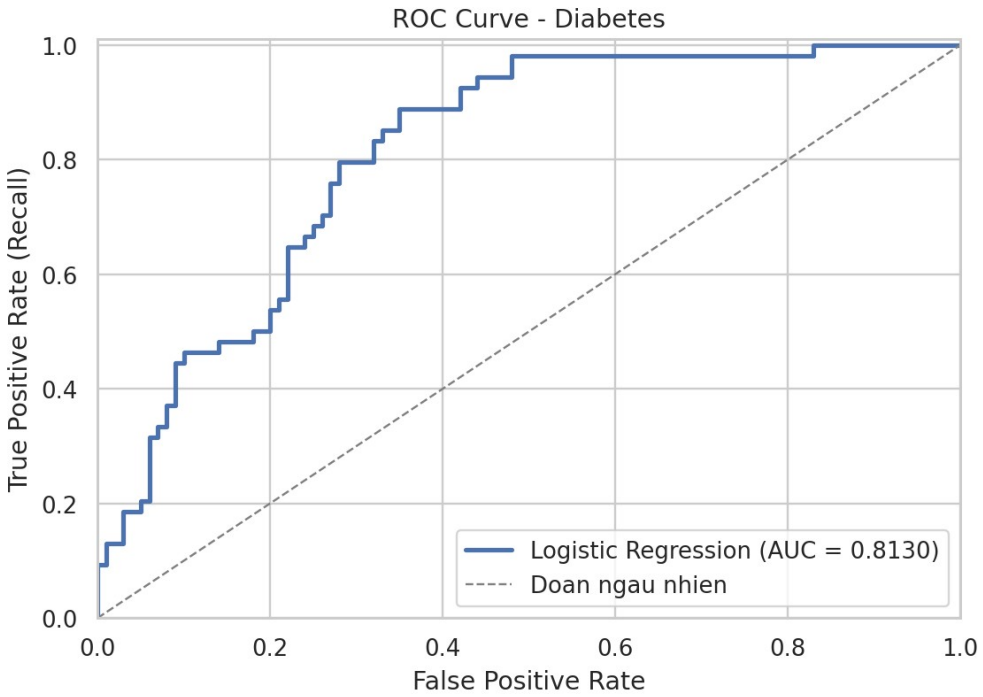
Trong tập dữ liệu: 500 người nhãn 0 và 268 người nhãn 1; sau stratified split, tập Test có 54 ca nhãn 1/154 người (35,06%).

Accuracy	Precision (bệnh)	Recall (bệnh)	F1 (bệnh)	ROC-AUC
0.7078	0.6000	0.5000	0.5455	0.8130

Confusion Matrix theo thứ tự [[TN, FP], [FN, TP]] = [[82, 18], [27, 27]]. Có 27 ca bệnh bị bỏ sót và 18 ca khỏe mạnh bị báo dương tính giả tại ngưỡng 0,5.



Hình 4. Ma trận nhầm lẫn trên 154 mẫu Test tại ngưỡng phân loại 0,5.

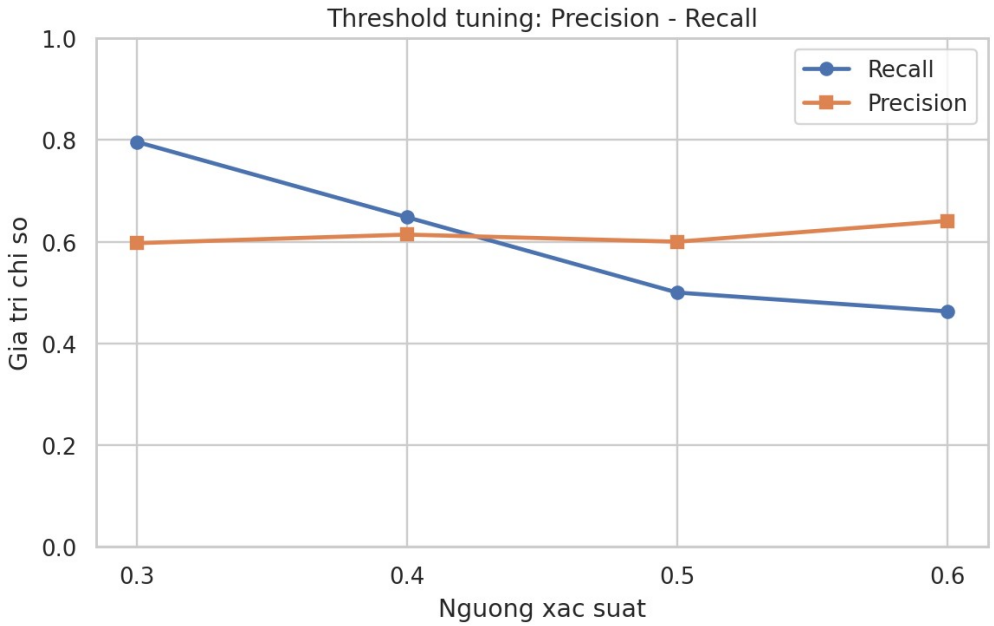


Hình 5. Đường cong ROC; diện tích AUC = 0.8130.

ROC-AUC xấp xỉ 0,813 cho thấy mô hình có khả năng phân biệt hai nhóm khá tốt trên bộ Test, song Recall 0,5 tại ngưỡng mặc định vẫn bỏ sót một nửa số trường hợp mang nhãn bệnh trong tập kiểm thử.

2.3. Thực nghiệm thay đổi ngưỡng quyết định

Ngưỡng	FP	FN	TP	Precision	Recall	F1
0.3	29	11	43	0.5972	0.7963	0.6825
0.4	22	19	35	0.6140	0.6481	0.6306
0.5	18	27	27	0.6000	0.5000	0.5455
0.6	14	29	25	0.6410	0.4630	0.5376



Hình 6. Precision – Recall khi dịch chuyển ngưỡng từ 0,3 đến 0,6.

So với ngưỡng 0,5, ngưỡng 0,3 giảm số FN từ 27 xuống 11 (ít hơn 16 ca), nâng Recall từ 0,5000 lên 0,7963, đồng thời tăng FP từ 18 lên 29. Vì mục tiêu minh họa là sàng lọc ưu tiên phát hiện ca bệnh, ngưỡng 0,3 là lựa chọn hợp lý nhất trong bốn mốc khảo sát. Tuy nhiên, lựa chọn ngưỡng trực tiếp trên Test làm thiên lệch đánh giá; quy trình thực tế cần chọn bằng Validation/CV trước khi mở Test cuối cùng.

III. CÂU HỎI BÁO CÁO THU HOẠCH

1. Cơ chế rò rỉ dữ liệu (Data Leakage)

Vấn đề: Nếu gọi scaler.fit_transform(X) trước train_test_split, giá trị trung bình và độ lệch chuẩn được tính từ toàn bộ mẫu, bao gồm 20% Test. Khi đó hàm biến đổi áp dụng cho Train đã sử dụng thông tin phân phối của những quan sát đáng lẽ chưa xuất hiện ở giai đoạn học. Đó là rò rỉ dữ liệu trong khâu tiền xử lý.

Ví dụ thực nghiệm: mean(area) trên Train của Housing bằng 5.154,14; trên toàn bộ 545 mẫu bằng 5.150,54. Với Diabetes sau khi bù 0 bằng median Train, mean(Glucose) của Train bằng 121,6710 còn toàn bộ Train+Test bằng 121,6563. Hai cách chuẩn hóa vì vậy khác nhau về thông số, dù chênh lệch trong dữ liệu này nhỏ.

Ảnh hưởng đến đánh giá: khi mô hình được chọn hoặc huấn luyện phụ thuộc vào thông tin từ Test, phép đo trên Test không còn hoàn toàn mô phỏng dữ liệu tương lai. Điều này có thể tạo ra ước lượng hiệu năng lạc quan và làm hỏng tính độc lập của quy trình đánh giá; mức độ tác động không nhất thiết lớn hay luôn tăng điểm trong mọi bài toán.

Đối chứng cụ thể: với bài Housing dùng LinearRegression không điều chuẩn và có intercept, chỉ thay bộ scaler Train bằng scaler toàn bộ dữ liệu cho ra $R^2 \approx 0{,}652924$ ở cả hai trường hợp (do mô hình tuyến tính không điều chuẩn bất biến trước phép đổi thang tuyến tính khả nghịch). Với Diabetes hiện tại, việc cho scaler nhìn cả Test cũng không làm thay đổi ROC-AUC làm tròn bốn chữ số: ≈ 0,8130. Đây KHÔNG phải bằng chứng cách làm sai là an toàn: ở mô hình điều chuẩn, lựa chọn đặc trưng, bù khuyết, cross-validation hay phân phối khác, rò rỉ có thể thay đổi kết quả đáng kể.

Cách khắc phục: luôn split trước; thực hiện fit Imputer/Scaler/Encoder học thống kê trên Train; transform trên Validation/Test. Khi dùng cross-validation, đặt tiền xử lý bên trong Pipeline để từng fold chỉ fit trên phần

Train tương ứng. Mọi siêu tham số và ngưỡng cũng phải chọn trên Train/Validation, không được thử lặp lại trên Test.

2. Đánh đổi Precision – Recall trong y tế

False Negative (FN) là người thực sự mang nhãn bệnh bị dự đoán khỏe mạnh; họ có thể bị trì hoãn thăm khám và điều trị, do đó trong một hệ thống sàng lọc thông thường đây là sai số cần ưu tiên giảm. False Positive (FP) khiến người khỏe mạnh cần kiểm tra bổ sung, gây chi phí và lo lắng nhưng thường ít nghiêm trọng hơn FN. Mức độ đánh đổi vẫn tùy độ tin cậy của xét nghiệm xác nhận và khả năng tiếp nhận của cơ sở y tế.

Recall = TP/(TP+FN) đo tỷ lệ bệnh nhân được phát hiện; Precision = TP/(TP+FP) đo tỷ lệ thật sự có bệnh trong các dự báo dương. Khi giảm ngưỡng từ 0,5 xuống 0,3: TP tăng 27 → 43, FN giảm 27 → 11 và Recall tăng 50,00% → 79,63%. Precision gần tương đương (60,00% → 59,72%) trong khi FP tăng 18 → 29. Đây là ưu điểm đáng kể cho nhiệm vụ sàng lọc.

Kiến nghị thực hành: chọn ngưỡng thử nghiệm 0,3 trong số bốn ngưỡng được giao vì số FN thấp nhất (11) và F1 cao nhất (0,6825). Không dùng kết quả như công cụ chẩn đoán lâm sàng. Khi triển khai thật, cần xác định chi phí FN/FP, độ nhạy tối thiểu, hiệu chỉnh xác suất, kiểm định ngoài mẫu và chọn ngưỡng trên một tập Validation riêng; sau đó kiểm tra một lần trên Test độc lập.

3. Ý nghĩa của hệ số điều chuẩn C

Trong LogisticRegression của scikit-learn, C > 0 là nghịch đảo độ mạnh regularization; với điều chuẩn L2, có thể viết hàm mục tiêu theo dạng tương đương: $J(w,b) = \text{Loss_logistic}(w,b) + \lambda\|w\|_2^2/2$, với λ tỷ lệ nghịch C (quy ước chuẩn hóa hằng số phụ thuộc cách định nghĩa Loss). Scikit-learn huấn luyện L2 theo mặc định. Thông thường intercept không bị phạt bởi solver lbfgs.

Nếu $C \to 0^+$, điều chuẩn mạnh: mô hình ưu tiên các trọng số nhỏ, ranh giới quyết định đơn giản/ít nhạy dữ liệu, phương sai giảm nhưng nguy cơ Underfitting tăng. Nếu $C \to +\infty$, điều chuẩn yếu dần, trọng số có thể lớn và mô hình bám sát dữ liệu huấn luyện hơn; nguy cơ Overfitting tăng trong dữ liệu nhiễu hoặc nhiều chiều. C không trực tiếp tăng số đặc trưng, bậc đa thức hay số lượng tham số của logistic tuyến tính.

Thực nghiệm giữ nguyên cách chia và biến đầu vào, chỉ đổi C; số liệu dưới đây là quan sát trên Train và Test, không phải quy trình tối ưu C bằng Test:

C	$\|w\|_2$	AUC Train	AUC Test
0.01	0.6832	0.8465	0.7985
0.1	1.2458	0.8509	0.8104
1	1.4486	0.8502	0.8130
10	1.4765	0.8500	0.8130
100	1.4794	0.8500	0.8131

Dữ liệu thực nghiệm cho thấy $\|w\|_2$ tăng từ 0,6832 (C=0,01) lên 1,4794 (C=100), phù hợp kỳ vọng yếu điều chuẩn hơn thì trọng số lớn hơn. AUC Test dao động 0,7985–0,8131, không cho thấy khoảng cách quá lớn tại dữ liệu này. Không thể khẳng định C=100 tốt hơn C=1 chỉ vì vài phần mười nghìn AUC trên một Test set; để chọn C cần cross-validation trên Train và đánh giá Test độc lập một lần.

IV. KẾT LUẬN & HƯỚNG PHÁT TRIỂN

Đã hoàn thành cả hai bài toán: biến đổi các cột định tính về dạng số; xử lý số 0 bất hợp lý bằng median Train; chuẩn hóa không rò rỉ dữ liệu; huấn luyện LinearRegression và LogisticRegression; thống kê MAE, RMSE,

R^2, Precision, Recall, F1, ROC-AUC; trực quan hóa phần dư, hệ số, Confusion Matrix, ROC và thử nghiệm thay đổi ngưỡng.

Kết quả chính: hồi quy nhà ở $R^2 = 0.6529$, MAE = 970,043; Logistic Regression AUC = 0.8130. Ở nhiệm vụ sàng lọc Diabetes, ngưỡng 0,3 cho Recall = 0.7963 và FN = 11, phù hợp ưu tiên hạn chế bỏ sót bệnh hơn ngưỡng mặc định 0,5.

Hướng phát triển: sử dụng Pipeline/ColumnTransformer và cross-validation trên Train; chọn ngưỡng theo chi phí/độ nhạy định trước; kiểm định phương sai sai số và outlier ở Housing; thử Elastic Net/Ridge/LogisticRegressionCV; đánh giá khả năng khái quát hóa bằng nguồn dữ liệu độc lập. Mọi kết quả Diabetes chỉ phục vụ mục đích học thuật.

PHỤ LỤC: CÁCH CHẠY TRÊN VISUAL STUDIO CODE

```
py -3 -m venv ml_lab_env
.\ml_lab_env\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python run_all.py
python -m unittest discover -s tests -v
```

Các notebook: notebooks/01_linear_regression.ipynb và notebooks/02_logistic_regression.ipynb. Mã Python: src/analysis.py và run_all.py. Biểu đồ và số liệu gốc tự động lưu trong outputs/ (6 hình PNG, các bảng CSV và experiment_results.json). Tập nguồn và các bước tái lập có trong README.md.

Tài liệu nền: Đề bài “Lab04-Linear Regression Logistic Regression.pdf” và 02 bộ dữ liệu CSV được cung cấp; bố cục báo cáo dựa trên tệp Word mẫu “65130011_Huynh_Trieu_An_BaoCao_ThucHanh04_CSP.docx”.